

Device Drivers

1 Basic Idea

Definition

A **device driver** is device-specific operating-system code that controls an I/O device by issuing commands to its controller and interpreting its status.

- Different devices have different registers, commands, timing rules, and status meanings.
- A mouse driver handles movement and button information.
- A disk driver handles sectors, tracks, cylinders, heads, arm movement, motor timing, and head settling.
- Because devices differ so much, each device type usually needs its own driver.

Core Point

The driver hides device-specific hardware details from the rest of the operating system.

2 Position in the System

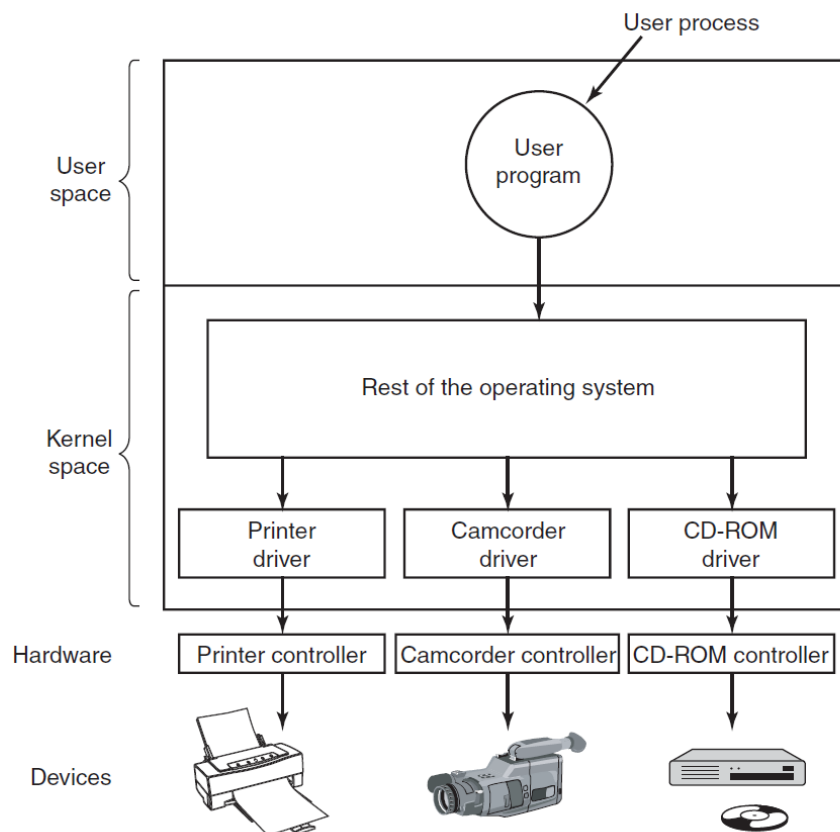


Figure 5-12. Logical positioning of device drivers. In reality all communication between drivers and device controllers goes over the bus.

Device drivers sit below the rest of the operating system. User programs call the operating system, not the driver directly. The rest of the OS sends abstract requests to drivers, and drivers translate those requests into controller commands.

3 One Driver or Many Drivers

Case	Explanation
One device type	A driver normally handles one device type.
Related device class	One SCSI disk driver may handle many SCSI disks of different sizes and speeds.
Unrelated devices	One driver could control unrelated devices, but it is usually a bad design.
Very different devices	A mouse and joystick usually need different drivers.

Design Rule

Putting unrelated devices into one driver makes the driver harder to maintain, test, and reason about.

4 Stacked Drivers and USB

USB Example

USB devices are very different at the top level, but they share lower-level USB communication layers.

Layer	Role in USB-style driver stack
Device-specific API	Provides behavior for mass storage, cameras, keyboards, and other device classes.
Common USB layer	Handles common USB packet and protocol functionality.
USB link layer	Handles low-level signaling and conversion between signals and USB packets.

5 Kernel Drivers vs. User-Space Drivers

Location	Benefit	Cost
Kernel space	Direct access to controller registers and efficient interaction with kernel services.	Buggy drivers can crash or corrupt the kernel.
User space	Better isolation between drivers and the kernel; improves reliability.	Needs system-call support to access registers and may add overhead.

6 Driver Interfaces

Standard Interface

The operating system defines standard procedures that drivers must support so the rest of the OS can call them uniformly.

Driver type	Device examples	Typical operation
Block driver	Disks, SSDs, optical drives	Read or write independently addressable blocks.
Character driver	Keyboards, printers, serial devices	Read or write streams of characters.

7 Main Driver Functions

Function	Meaning
Accept requests	Receive abstract read and write requests from device-independent software.
Validate input	Check that request parameters are legal.
Translate requests	Convert abstract requests into concrete device terms.
Initialize device	Prepare the device before use if needed.
Manage power	Turn devices on, off, or into low-power states.
Issue commands	Write command sequences into controller registers.
Handle completion	Block for interrupts or complete immediately when no wait is needed.
Check errors	Examine controller status and report success or failure.
Start queued work	Select and start another waiting request when possible.

8 Typical Driver Workflow

1. Receive a request from device-independent OS software.
2. Check whether input parameters are valid.
3. Translate the request into device-specific terms.
4. Queue the request if the device is already busy.
5. If idle, examine hardware status and power on the device if required.
6. Issue the required command sequence to controller registers.
7. Wait for completion if the operation takes time.
8. Check for errors, pass data upward, and return status.
9. Start another queued request, or block waiting for the next request.

9 Why Drivers Are Complicated

Reentrant Drivers

A driver may be interrupted while it is already running, and the interrupt may cause that same driver to run again. Shared driver data structures must be protected carefully.

Hot-pluggable devices add another problem: a device may be removed while the driver is performing I/O. The driver must abort the current transfer, cancel pending requests, and tell callers that the device is gone.

Kernel Interaction

Drivers normally cannot make ordinary system calls, but they can call selected kernel procedures for pinned memory, timers, DMA, interrupt control, logging, and error reporting.

Final Point

Device drivers are the boundary between general OS I/O code and device-specific hardware behavior.